App Dev Project Report

1. Student Details

Name: Priyanshi Vasudev
Roll Number: 23F2005277
Email: 23f2005277@ds.study.iitm.ac.in
About Me: I am a physics honours graduate and going to pursue masters in the same. I enjoy solving analytical problems.

2. Project Details

Project Title: Hospital Management System

Problem Statement:
To design and develop a web-based application that allows efficient management of hospital operations including doctor management, patient registration, appointment booking, treatment tracking, and administrative control.

Approach:
The application is built using a **Flask backend** and **Vue 3 frontend**. It follows a REST API-based architecture where the frontend communicates with the backend using HTTP requests. The system supports role-based access (Admin, Doctor, Patient), appointment scheduling with time slots, and treatment record management.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) and Claude for assistance in:

- Understanding concepts (Flask, Vue, JWT)
- Debugging errors
- Improving code structure and explanations

The extent of AI usage is approximately **15%**, mainly for guidance and learning purposes.
All implementation, logic building, and integration were done manually.

4. Technologies and Frameworks Used

Frontend

Technology	Purpose
Vue 3	UI Framework
Vite	Build Tool

Vue Router 4	Routing
Axios / Fetch	HTTP Client
CSS	Styling

Backend

Technology	Purpose
Flask	Web Framework
SQLAlchemy	ORM
SQLite	Database
JWT	Authentication
RBAC	Authorization
CORS	Cross-Origin
Flask-Caching (Redis)	Performance

5. Database Schema / ER Diagram

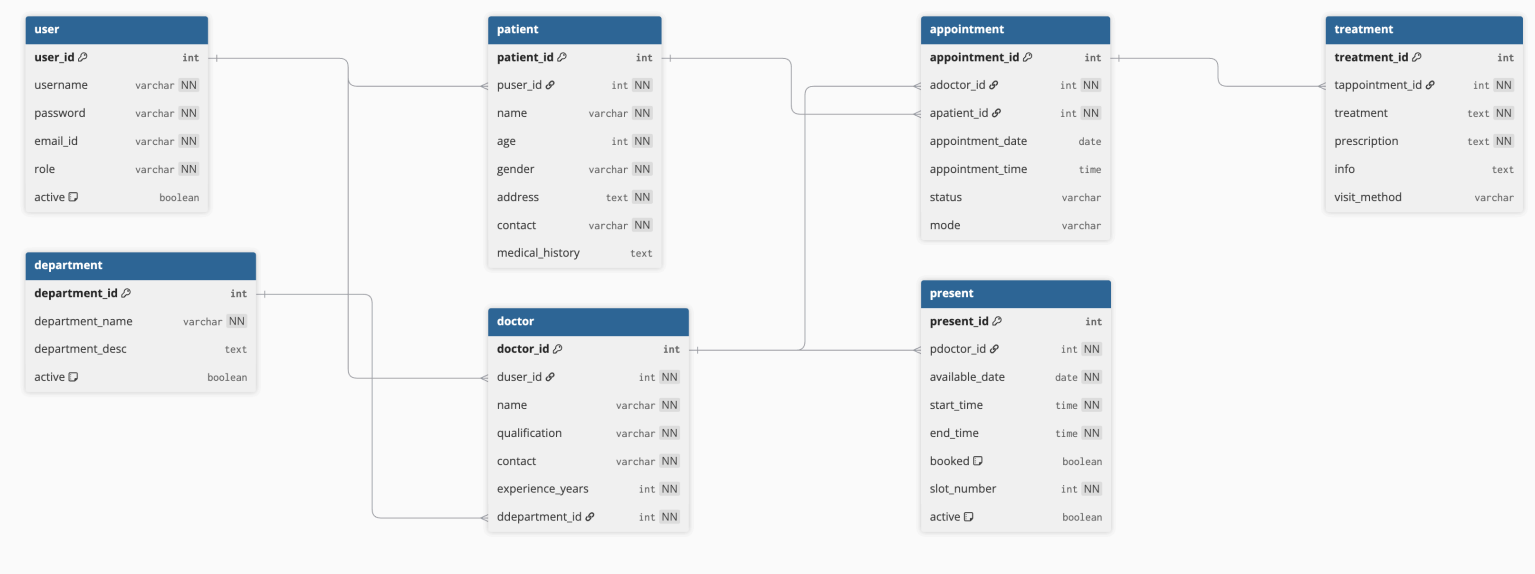
Tables:

1. **User** — stores login and role details
2. **Doctor** — doctor-specific information
3. **Patient** — patient profile details
4. **Department** — hospital departments
5. **Appointment** — booking information
6. **Treatment** — treatment and prescription details
7. **Present** — doctor availability slots

Relationships:

- One-to-One → User → Doctor
- One-to-One → User → Patient
- One-to-Many → Department → Doctor
- One-to-Many → Doctor → Appointment

- One-to-Many → Patient → Appointment
- One-to-One → Appointment → Treatment
- One-to-Many → Doctor → Present



6. API Resource Endpoints

Endpoint	Method	Description
/api/register	POST	Register new patient
/api/login	POST	User login with JWT
/api/add_doctor	POST	Add doctor (Admin only)
/api/alldoctors	GET	Get all doctors
/api/add_department	POST	Add department
/api/alldepartments	GET	Get departments
/api/patient/select_slot	GET	View available slots
/api/patient/book_slot	PUT	Book appointment
/api/doctor/allappointments	GET	Doctor appointments
/api/doctor/update_treatment	PUT	Add/update treatment
/api/admin/allappointment	GET	Admin view all appointments
/api/patient/export_data	POST	Export patient data

7. Architecture and Features

Architecture Overview:

- **app.py** – Main backend file containing API routes
- **models.py** – Database models using SQLAlchemy
- **Frontend (Vue 3)** – Handles UI and user interaction
- **API Layer** – Connects frontend and backend using Axios
- **Database (SQLite)** – Stores application data

Implemented Features:

- User authentication using JWT
- Role-based access control (Admin, Doctor, Patient)
- Doctor and department management
- Appointment booking with time slots
- Slot activation/deactivation by doctors
- Appointment cancellation (doctor & patient)
- Treatment and prescription management
- Patient medical history tracking
- Search functionality (doctor/patient)
- API caching using Redis
- Export patient data (background task support)

Additional Features:

- Automatic slot generation for next 7 days
- Prevention of double booking
- Secure password hashing
- Soft delete using active flag

8. Video Presentation

Drive Link: https://drive.google.com/file/d/1cpwBoAoZ7reyXDhtQxz8cG06l0Dq7YF5/view?usp=sharing